

# Starting A Project

---

Regardless of the methodology being used, *certain steps and documentations are NECESSARY.*

Although you may go *back and forth* between certain **CORE PROCESSES** of creating your application.

- You always have to start with **ENVISIONING your application**, *with respect to the client you are building it for.*

For that, one **KEY** document we use, is a System Visions Document (SVD)

---

Let's say a library in your town is still using a *manual paper accounting system.*

> They tell you:

*"We print cards to track checkout's and due dates, manually calculate fines off of them. Sometimes we calculate fines incorrectly or lose cards, resulting in customers being unhappy; as well as our inline staff waste hours on auditing the system manually."*

What we need to do is answer a couple questions, so we can create a **central document (SVD)** on what *our application* is planning to provide, and *why that matters to them*

- What **EXACTLY** is the **problem?** (1.1)
  - How can they **BENEFIT** from the **issues the system solves?** (1.2)
  - What **EXACTLY** can the **system do?** (1.3)
  - What **STANDS** in the way of the **system?** (1.4)
- 

> 1.1 What **EXACTLY** is the **problem?**

This part is quite simple, take what the client told you, and *transform it into a problem statement.*

> Take what they told us, **transform it into a PROBLEM to solve.**

- *"Manual accounting system where the client print cards for bookkeeping sometimes get lost; manual fines calculations are sometimes inaccurate. Resulting in waste of staff*

*time for menial tasks, and unhappy customers."*

## > 1.2 How can they BENEFIT from the **issues the system solves?**

What we want to do here, is *take the issues we picked out in 1.1, and attach value to solving them.*

| Problem                           | Benefit               | Value Type | Value                          |
|-----------------------------------|-----------------------|------------|--------------------------------|
| "Books go missing"                | Reduce lost inventory | Tangible   | <b>Estimate on books saved</b> |
| "Patrons complain about fines"    | Fewer disputes        | Tangible   | <b>Staff time saved</b>        |
| "Hours each week on manual tasks" | Staff time saved      | Tangible   | <b>Observed time save</b>      |
| "Should be automated"             | Modernization         | Intangible | <b>Better image</b>            |

> This better helps *paint the picture* on what we're providing, and what **use it has.**

---

## > 1.3 What EXACTLY *can* the **system do?**

Now, we want to say WHAT exactly the client can **expect** *the system to do.*

*Track Book Checkouts > Record when books are checked out to patrons*

*Stamp Due Dates > Automatically track and stamp due dates*

*Calculate Fines > Track overdue items, authority on fines*

> What we did here, was match the benefit to *how the system* will let them **ATTAIN those benefits.**

---



## > 1.4 What STANDS in the way of the system?

Here, we want to ask specific questions, on *requirements for the system*. Then provide an answer to them.

### > Organizational

RAISES a social question with the system:

- (Resistance to change, fear of technology, willingness to learn new processes) → "Management will provide training."

### > Technological

RAISES a technical issue:

- (Existing infrastructure, Compatibility, Technical complexity) → "Library has Wi-Fi and computers for the system to function."

### > Resources

RAISES a technical issue:

- (Budget, Training staff, Amount of developers they can afford) → "\$75,000 budget approved with 2 aid staff and 3 developers."

### > Schedule

RAISES schedule issue:

- (Need this by this summer, need the BASE SYSTEM by next year) → "Project timeline set for 4 months"

---

In the end, we bring it all together, so BOTH we and the client can be on the same page on what **they're getting** and what **we're making**.

- One key thing to note is, this might be for the ENTIRE project (Waterfall), or just for a small subsection of it (Agile)
- In the case of Agile, *budget and timing* are seen to be a "best guess" not necessarily law.

*That's actually it for SVD, next part will focus on REQUIREMENTS of a system*

→ See next page for full SVD.

# Client Library SVD

## 1.1 Problem Statement

*Manual accounting system where staff print cards for bookkeeping that sometimes get lost; manual fine calculations are sometimes inaccurate. Results in wasted staff time on menial tasks and unhappy customers.*

## 1.2 Business Benefits

Problem: Books go missing

Benefit: Reduce lost inventory

Type: Tangible

Value: \$8,000/year in books saved

Problem: Patrons complain about fines

Benefit: Fewer disputes

Type: Tangible

Value: 5 staff hours/week saved

Problem: Hours on manual tasks

Benefit: Staff time saved

Type: Tangible

Value: 15 staff hours/week saved

Problem: Manual system outdated

Benefit: Modernization

Type: Intangible

Value: Improved library image

## 1.3 System Capabilities

Track Checkouts – Record when books are checked out to patrons

Due Dates – Automatically calculate and stamp due dates

Calculate Fines – Track overdue items as single source of truth for fines

Patron Management – Maintain patron accounts and borrowing history

Search Catalog – Allow patrons to find books by title/author

## 1.4 Feasibility Assessment

☐ 6: Will staff use it? → Management will provide training

☐ **Technological:** Can we build it? → Library has WiFi and computers

☐ **Resource:** Can we afford it? → \$75,000 budget with 2 staff and 3 developers

☐ **Schedule:** Can we deliver on time? → 4-month timeline is achievable

